

Describe Laboratorio #3 - Cassandra

Documentación Oficial: <https://cassandra.apache.org/doc/latest/>

Introducción: Apache Cassandra es una base de datos NoSQL distribuida orientada a columnas. Este laboratorio introduce CQL, modelado orientado a consultas, colecciones, UDTs, Tuples y Vectors.

A diferencia de las bases de datos relacionales, Cassandra se basa en el concepto de:

- Keyspaces (similares a bases de datos):
- Tables (similares a tablas)
- Partition Keys
- Clustering Keys

Algunos Comandos Básicos

Ingresar al cliente CQL → cqlsh

```
Connected to Test Cluster at 127.0.0.1:9042
[cqlsh 6.2.0 | Cassandra 5.0.0 | CQL spec 3.4.7 | Native protocol v5]
Use HELP for help.
cqlsh>
```

La terminal quedará a la espera de que ingreses comandos en lenguaje CQL.

Consultar versión → SHOW VERSION;

```
cqlsh> SHOW VERSION;
[cqlsh 6.2.0 | Cassandra 5.0.0 | CQL spec 3.4.7 | Native protocol v5]
```

Muestra la versión del cliente de cqlsh, la versión de Apache Cassandra a la que estamos conectados y la especificación del protocolo CQL.

Consultar los KeySpaces existente → DESCRIBE KEYSPACES;

```
cqlsh> DESCRIBE KEYSPACES;

system_schema system_auth system system_distributed system_traces
mi_aplicacion
```

Muestra los *keyspaces* del sistema (tienen la palabra system) y los de los usuarios (mi_aplicacion).

Creación de Keyspace → CREATE KEYSPACE

```
CREATE KEYSPACE nombre
WITH REPLICATION = 'class':'SimpleStrategy' , 'replication_factor':1};
```

Selección de un KeySpace existente → USE

USE nombre_keyspace;

Tipos de Datos Basicos

Tipo	Tamaño / Rango	Descripción
tinyint	1 byte (-128 a 127)	Entero de 8 bits firmado.
smallint	2 bytes (-32,768 a 32,767)	Entero de 16 bits firmado.
int	4 bytes (-2^{31} a $2^{31}-1$)	Entero de 32 bits firmado (estándar).
bigint	8 bytes (-2^{63} a $2^{63}-1$)	Entero de 64 bits firmado.
varint	Tamaño variable	Entero de precisión arbitraria.
float	4 bytes	Número de punto flotante de precisión simple
double	8 bytes	Número de punto flotante de doble precisión
decimal	Variable	Número decimal de precisión exacta (moneda)
counter	8 bytes	Entero especial de 64 bits cuyo valor solo puede incrementarse o decrementarse. No se puede escribir directamente con INSERT.
boolean	Valor lógico true o false.	
blob	Datos binarios no estructurados en formato hexadecimal (útil para imágenes pequeñas, archivos o serializaciones).	
inet	Dirección IP en formato IPv4 (ej. '192.168.1.1') o IPv6.	
timestamp	Fecha y hora guardadas como un entero de 64 bits (milisegundos desde la época Unix). Permite ingresar datos en formato ISO (ej. '2026-09-06 12:00:00+0000').	
date	Fecha sin hora (formato YYYY-MM-DD). Internamente es un entero de 32 bits.	
time	Hora del día sin fecha con precisión de nanosegundos desde la medianoche.	
duration	Lapso de tiempo expresado en meses, días y nanosegundos (ej. 3d2h o 1mo2d).	
uuid	Identificador Único Universal de 128 bits (Estándar RFC 4122). Evita colisiones de claves primarias en clústeres distribuidos.	
timeuuid	UUID Versión 1. Incluye la marca de tiempo (timestamp) del momento en que se generó, lo que permite ordenar los registros cronológicamente.	
text	Cadena de texto codificada en UTF-8 (es el tipo recomendado y equivalente a varchar).	
varchar	Alias de text (funciona exactamente igual).	
ascii	Cadena de texto limitada a caracteres ASCII.	

Colecciones (LIST<T>, SET<T>, MAP<K, V>)

Permiten almacenar múltiples valores en una sola celda/columna sin necesidad de crear tablas secundarias normalizadas:

- List<type>: Colección ordenada de elementos que permite duplicados. Preserva el orden de inserción.
emails list<text> → ['a@test.com', 'b@test.com']
- Set<type>: Colección de elementos sin duplicados. Se devuelven en orden.
tags set<text> → {'java', 'nosql', 'cassandra'}
- Map<type1, type2>: Colección de pares clave-valor (diccionario o hash).
scores map<text, int> → {'math': 90, 'physics': 85}

Tipos avanzados y personalizados

User-Defined Types (UDT): Permiten definir una estructura personalizada mediante el comando CREATE TYPE. Son útiles para agrupar campos relacionados en una sola columna.

```
CREATE TYPE direccion (  
    calle text,  
    ciudad text,  
    codigo_postal int  
);  
  
CREATE TABLE usuarios (  
    id uuid PRIMARY KEY,  
    nombre text,  
    domicilio frozen<direccion>  
);
```

¿Cuándo conviene usar UDT?

¿Qué hace el modificador frozen?

¿Qué ventajas tiene su uso?

Tuplas: Colección de valores anónimos y de longitud fija que pueden tener diferentes tipos de datos entre sí. Se declaran como frozen implícitamente (ej. tuple<text, int, boolean>).

Ejemplo: Estamos diseñando el modelo de datos para una plataforma de educación online (un "LMS" estilo Moodle o Google Classroom) y queremos registrar las entregas de los trabajos prácticos que realizan los alumnos.

Para cada entrega necesitamos registrar:

- Las coordenadas GPS (Latitud, Longitud) desde donde el alumno envió el trabajo (para validar presencia o evitar fraudes en exámenes presenciales).
- La calificación expresada como una nota numérica y una letra (ej. (8.5, 'B+')).
- Los datos del Alumno

```
CREATE TABLE entregas_tp (  
    materia_id uuid,  
    materia_nombre text,  
    alumno_id uuid,  
    alumno_nombre text,  
    tp_id int,  
    fecha_entrega timestamp,  
    ubicacion_gps tuple<double, double>,  
    calificacion tuple<decimal, text>,  
    PRIMARY KEY ((materia_id, tp_id), alumno_id)  
);
```

Los valores de la tupla se ingresan entre paréntesis (...) respetando el orden y tipo de dato declarado:

```

INSERT INTO entregas_tp (
    materia_id,
    materia_nombre,
    tp_id,
    alumno_id,
    alumno_nombre,
    fecha_entrega,
    ubicacion_gps,
    calificacion
) VALUES (
    a3b8e8f0-1234-11ed-a100-0242ac120002,
    'Ingeniería de Datos II',
    1,
    c9d8e7f0-5678-11ed-a100-0242ac120002,
    'Cacho Gomez',
    toTimestamp(now()),
    (-34.6037, -58.3816), -- (Latitud, Longitud de CABA)
    (8.50, 'Aprobado')
);

```

Decido cambiar el resultado, ya que tiene más de 8 esta promocionado.

```

UPDATE entregas_tp
SET calificacion[1] = 'Promocionado'
WHERE materia_id = a3b8e8f0-1234-11ed-a100-0242ac120002
    AND tp_id = 1
    AND alumno_id = c9d8e7f0-5678-11ed-a100-0242ac120002;

```

¿Por qué no funciona?

¿Cómo debería escribirse correctamente?

Vectores (VECTOR):

Un vector en este contexto no es más que una lista ordenada de números decimales de longitud fija (también llamados flotantes). Cada número dentro del vector representa una dimensión específica en un espacio abstracto multidimensional.

La declaración **VECTOR<FLOAT, 5>** define:

- **FLOAT:** El tipo de número que almacena cada posición (números con decimales).
- **5:** La cantidad exacta de elementos (dimensiones). Todos los registros de la columna deberán tener exactamente 5 números.

Los humanos procesamos texto ("Introducción a Cassandra"), pero los modelos de Inteligencia Artificial (como los de OpenAI, Hugging Face o Cohere) procesan números.

Un modelo de IA toma un contenido (texto, imagen, audio) y lo traduce a una lista de números conocida como Embedding.

Significado Semántico: El modelo asigna estos números de forma que conceptos conceptualmente parecidos terminen con números similares o cercanos en el espacio multidimensional.

Dimensiones reales: En proyectos reales de producción, en lugar de 5 dimensiones, los vectores suelen tener 768, 1536 o 3072 dimensiones, dependiendo del modelo de IA utilizado.

A diferencia de una consulta SQL tradicional donde buscas coincidencias exactas (WHERE title = 'Cassandra'), con un tipo de dato VECTOR buscas por similitud semántica o significado.

Búsqueda Semántica

Si un usuario busca "Aprender base de datos NoSQL de Apache", una búsqueda tradicional por palabra exacta podría fallar si la palabra "NoSQL" no está en el título. Con vectores:

- 1) Conviertes la consulta del usuario a un vector.
- 2) Calculas la distancia geométrica (como Coseno o Distancia Euclidiana) entre el vector de la consulta y los vectores almacenados.
- 3) La base de datos devuelve los artículos conceptualmente más cercanos, aunque no compartan las mismas palabras.

Motores de Recomendación

Si a un usuario le gusta un artículo con un vector específico, la base de datos puede encontrar otros artículos cuyos vectores estén a menor distancia geométrica.

Arquitectura RAG (Retrieval-Augmented Generation)

Es la técnica principal usada para darle memoria o contexto de negocio a modelos de lenguaje (LLMs como ChatGPT). Almacenas la documentación de tu empresa como vectores y, cuando el usuario pregunta algo, la base de datos recupera la información más relevante antes de responder.

```
SELECT article_id, title, similarity_cosine(embedding, [0.15, 0.42, 0.88, 0.30, 0.20])
AS score
FROM articles
ORDER BY embedding ANN OF [0.15, 0.42, 0.88, 0.30, 0.20]
LIMIT 3;
```

ANN OF [...]: Le indica al motor que use la estructura de índice HNSW (Hierarchical Navigable Small World) para estimar de forma rápida cuáles son los vectores más cercanos sin comparar uno por uno frente a millones de filas.

Modelado de Consultas

- A) Se intenta modelar una liga con equipos, jugadores, temporadas, partidos y goleadores. El objetivo es diseñar tablas orientadas a satisfacer estas consultas e insertar datos para verificar los resultados.
- Partidos por Torneo: (todos los partidos del torneo ordenados desde el más reciente)
 - Partidos por Equipo: (historial completo del equipo)
 - Partidos por Equipo y Temporada: (únicamente encuentros de una temporada específica)
 - Últimos 10 partidos: (últimos 10 partidos de una temporada)
 - Ranking de Goleadores (jugadores ordenados por cantidad de goles)

En todos los casos contestar

¿qué columna/s es partition key?

¿qué debería cambiar en la consulta para que la table ya no fuese válida?

¿cómo impactaría en el tamaño de las particiones?

B) Se intenta modelar un sistema de recomendaciones de películas con IA. El objetivo es diseñar tablas orientadas a satisfacer estas consultas e insertar datos para verificar los resultados.

```
CREATE TABLE movies (  
  movie_id UUID PRIMARY KEY,  
  title TEXT,  
  genre TEXT,  
  year INT,  
  embedding VECTOR<FLOAT,8>  
);
```

Insertar 10 películas utilizando embeddings ficticios.

Realizar las siguientes tareas:

- Consultar todas las películas.
- Buscar películas por género.
- Buscar películas por año.
- Agregar una colección SET<TEXT> para almacenar actores.
- Agregar un MAP<TEXT,TEXT> con información adicional.
- Incorporar un UDT para director.
- Crear un índice sobre el vector

```
CREATE CUSTOM INDEX IF NOT EXISTS idx_movies_embedding  
ON movies (embedding) USING 'StorageAttachedIndex'  
WITH OPTIONS = {'similarity_function': 'COSINE'};
```

- Insertar embeddings utilizando VECTOR<FLOAT,8>.

```
INSERT INTO movies (movie_id, title, genre, year, embedding)  
VALUES (uuid(), 'Inception', 'Sci-Fi', 2010, [0.12, 0.45, 0.91, 0.33, 0.18, 0.50, 0.05, 0.82]);
```

Ejemplo de Select:

```
SELECT movie_id, title, genre, year,  
       similarity_cosine(embedding, [0.12, 0.45, 0.91, 0.33, 0.18, 0.50, 0.05, 0.82]) AS score  
FROM movies  
ORDER BY embedding ANN OF [0.12, 0.45, 0.91, 0.33, 0.18, 0.50, 0.05, 0.82]  
LIMIT 5;
```

Probar hacer otras búsquedas o cambiar el limite

- Explique cómo funciona esa sentencia Select. ¡Que hace cada parte?